

Shivam Kumar

Roll. No.: 22CE01046

November 24, 2025

GitHub: <https://github.com/username/shivamkumar>

Assignment-2 (CAD Laboratory)

Question 1. Given the two-dimensional body

$$\mathcal{B} = \{(X_1, X_2) | 0.1 < X_1 < 1, 0.1 < X_2 < 1\}$$

and the displacement field

$$u_1 = u \cdot e_1 = 0.2 \ln(1 + X_1 + X_2), u_2 = u \cdot e_2 = 0.2 \exp X_1.$$

Plot the displaced shape of the body using Julia.

Hint: First set up some characteristic lines on the body. For each point compute its displacement and add it to the original position by noting

$$x_i = e_i \cdot (X + u)$$

Solution: 1

```
1      using Plots
2
3      # 1. Define the original domain (undeformed grid)
4      x1_range = 0.1:0.05:1.0
5      x2_range = 0.1:0.05:1.0
6
7      # Lists to store coordinates
8      X1_orig, X2_orig = Float64[], Float64[]
9      x1_new, x2_new = Float64[], Float64[]
10
11     # 2. Compute displacement and new positions
12     for x1 in x1_range
13         for x2 in x2_range
14             # Original Coordinates
15             push!(X1_orig, x1)
16             push!(X2_orig, x2)
17
18             # Displacement field u1, u2 [cite: 16]
19             u1 = 0.2 * log(1 + x1 + x2)
20             u2 = 0.2 * exp(x1)
21
22             # Deformed Coordinates xi = Xi + ui
23             push!(x1_new, x1 + u1)
24             push!(x2_new, x2 + u2)
25     end
```

```

26     end
27
28     # 3. Plotting
29     scatter(X1_orig, X2_orig, label="Original Configuration", marker=:circle,
30           ↪ legend=:topleft)
31     scatter!(x1_new, x2_new, label="Deformed Configuration", marker=:cross)
32     title!("Q1: Displacement of Rectangular Body")
33     xlabel!("X1")
34     ylabel!("X2")

```

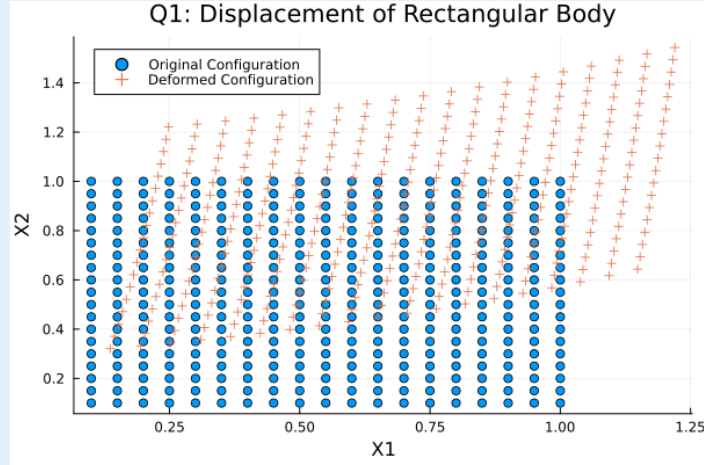


FIGURE 1. Displacement of Rectangular Body

Question 2. With respect to a two-dimensional rectangular cartesian coordinates system with orthonormal base vectors (e_1, e_2) , let the rectangular coordinates of a point be denoted by (X_1, X_2) . Consider a two-dimensional annular body $\mathcal{B}$ which occupies the region:

$$\mathcal{B} = \{(X_1, X_2) | 1 < \sqrt{X_1^2 + X_2^2} < 2\}$$

Because of the annular nature of the body consider a polar coordinates system in which the coordinates of points are expressed in terms (R, θ) , which are related to the rectangular coordinates (X_1, X_2) by:

$$R = \sqrt{X_1^2 + X_2^2} \text{ and } \theta = \tan^{-1}(X_2/X_1)$$

Further, the orthonormal base vectors (e_r, e_θ) of the polar coordinate system are related to (e_1, e_2) by:

$$e_r = \cos \theta e_1 + \sin \theta e_2, e_\theta = -\sin \theta e_1 + \cos \theta e_2$$

Draw the deformed configuration of the annular body associated with the following deformation using Julia:

$$u_r = u \cdot e_r = 0.4(R - 1)^2 \cos 3\theta$$

$$u_\theta = u \cdot e_\theta = 0.4(R - 1)^3$$

****Hint:**** First set up some characteristic lines on the body in terms of their (R, θ) coordinates. Compute the (X_1, X_2) coordinates from

$$X_1 = R \cos \theta, X_2 = R \sin \theta$$

. For each point compute its displacement and add it to the original position by noting $x_i = e_i \cdot (X + u) = e_i \cdot (X + u_r e_r + u_\theta e_\theta)$.

Solution: 2

```

1      using Plots
2
3      # 1. Define the annular domain using Polar Coordinates
4      R_range = 1.0:0.1:2.0
5      Theta_range = 0:0.01:2*pi
6
7      X1_orig, X2_orig = Float64[], Float64[]
8      x1_new, x2_new = Float64[], Float64[]
9
10     for r in R_range
11         for theta in Theta_range
12             # Convert Polar to Cartesian for Original Points [cite: 33]
13             X1 = r * cos(theta)
14             X2 = r * sin(theta)
15
16             push!(X1_orig, X1)
17             push!(X2_orig, X2)
18
19             # Calculate Displacements in Polar Basis [cite: 29]
20             u_r = 0.4 * (r - 1)^2 * cos(3 * theta)
21             u_theta = 0.4 * (r - 1)^3
22
23             # Transformation to Cartesian Basis
24             # u1 = u_r * cos(theta) - u_theta * sin(theta)
25             # u2 = u_r * sin(theta) + u_theta * cos(theta)
26             # Derived from basis vector relations [cite: 25, 34]
27
28             u1 = u_r * cos(theta) - u_theta * sin(theta)
29             u2 = u_r * sin(theta) + u_theta * cos(theta)
30
31             # Add displacement to original Cartesian coords
32             push!(x1_new, X1 + u1)
33             push!(x2_new, X2 + u2)
34         end
35     end
36

```

```

37  # 3. Plotting
38  scatter(X1_orig, X2_orig, label="Original", aspect_ratio=:equal, markersize=2)
39  scatter!(x1_new, x2_new, label="Deformed", markersize=2)
40  title!("Q2: Annular Body Deformation")
41
42  savefig("Annular_Body_Deformation.png")  # save the figure

```

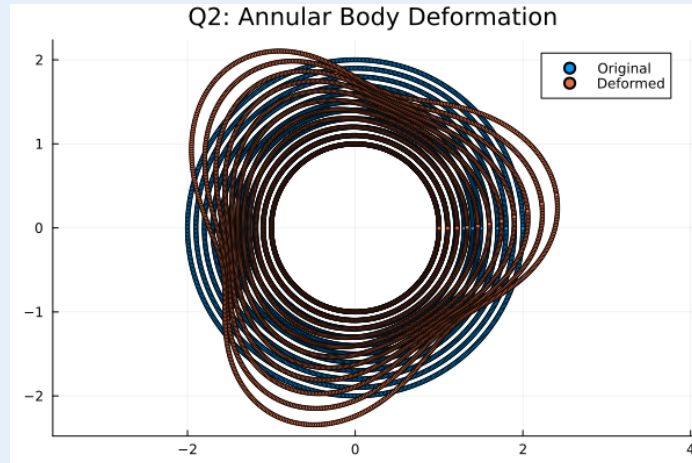


FIGURE 2. Annular Body Deformation

Question 3. Given the two-dimensional body

$$\mathcal{B} = \{(X_1, X_2) | 0.1 < X_1 < 1, 0.1 < X_2 < 1\}$$

and the displacement field

$$u_r = u \cdot e_r = 0.2 \exp X_1, u_\theta = u \cdot e_\theta = 0.2 \ln(1 + X_1 + X_2)$$

Plot the displaced shape of the body using Julia.

Solution: 3

```

1  using Plots
2
3  x1_range = 0.1:0.05:1.0
4  x2_range = 0.1:0.05:1.0
5
6  X1_orig, X2_orig = Float64[], Float64[]
7  x1_new, x2_new = Float64[], Float64[]
8
9  for X1 in x1_range
10     for X2 in x2_range
11         push!(X1_orig, X1)

```

```

12     push!(X2_orig, X2)
13
14     # Calculate local polar parameters for basis transformation [cite: 23]
15     R = sqrt(X1^2 + X2^2)
16     theta = atan(X2, X1)
17
18     # Calculate displacements (given values) [cite: 38, 40]
19     u_r_val = 0.2 * exp(X1)
20     u_theta_val = 0.2 * log(1 + X1 + X2)
21
22     # Convert to Cartesian components u1, u2 using local theta
23     # u = u_r * e_r + u_theta * e_theta
24     u1 = u_r_val * cos(theta) - u_theta_val * sin(theta)
25     u2 = u_r_val * sin(theta) + u_theta_val * cos(theta)
26
27     push!(x1_new, X1 + u1)
28     push!(x2_new, X2 + u2)
29 end
30 end
31
32 scatter(X1_orig, X2_orig, label="Original", legend=:topleft)
33 scatter!(x1_new, x2_new, label="Deformed")
34 title!("Q3: Mixed Basis Deformation")
35
36 savefig("Mixed_Basis_Deformation.png") # save the figure

```

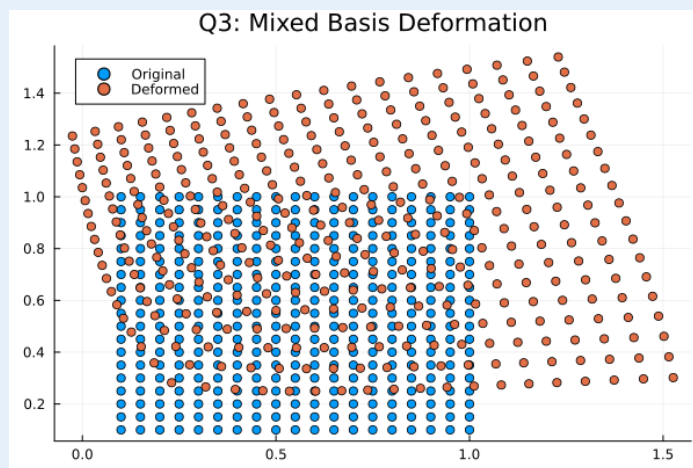


FIGURE 3. Mixed Basis Deformation

Question 4. Find out the stress distribution by modeling the given rectangular plate of 2 mm thick with a circular hole. The dimensions of the steel plate and hole are shown in Figure 1. The section is made up of steel having a value of 210 GPa for Young's modulus and a value of 0.3 for Poisson's ratio.

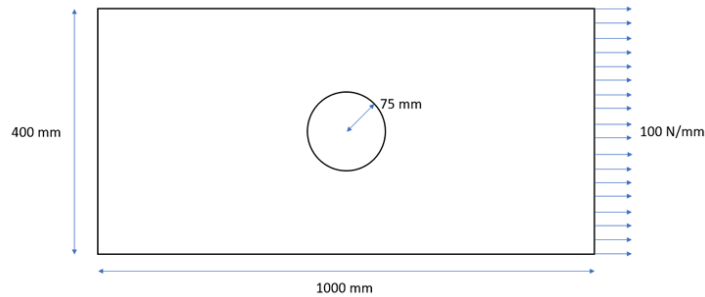


FIGURE 4. Caption

- Provide the steps for finite element meshing both in Julia and Abaqus.
- Mention the governing PDEs that are required to solve for the stress distribution.
- Provide a comparison between the results obtained from Gridap in Julia and Abaqus.

Solution: 4 (a) Steps for finite element meshing using Julia.

```

1      import Gmsh: gmsh
2
3      # Initializing Gmsh
4      gmsh.initialize()
5
6      gmsh.model.add("PlateHole_Q4")
7
8      # --- Parameters (Converted to Meters to match your L=1 scale) ---
9      L = 1.0      # 1000 mm
10     H = 0.4      # 400 mm
11     R = 0.075    # Radius 75 mm
12     Xc = 0.5     # Center X (500 mm)
13     Yc = 0.2     # Center Y (200 mm)
14
15     # Mesh sizes
16     h_coarse = 0.008 # Coarse mesh for far boundaries
17     h_fine   = 0.005 # Fine mesh for the hole (CRITICAL for stress concentration)
18
19     # --- 1. Define the Outer Rectangle ---
20     p1 = gmsh.model.geo.addPoint(0, 0, 0, h_coarse)
21     p2 = gmsh.model.geo.addPoint(L, 0, 0, h_coarse)
22     p3 = gmsh.model.geo.addPoint(L, H, 0, h_coarse)
23     p4 = gmsh.model.geo.addPoint(0, H, 0, h_coarse)
24

```

```

25     l1 = gmsh.model.geo.addLine(p1, p2)
26     l2 = gmsh.model.geo.addLine(p2, p3)
27     l3 = gmsh.model.geo.addLine(p3, p4)
28     l4 = gmsh.model.geo.addLine(p4, p1)
29
30     # Outer Loop
31     cl_rect = gmsh.model.geo.addCurveLoop([l1, l2, l3, l4])
32
33     # --- 2. Define the Inner Circle ---
34     # We create 5 points: Center + 4 points on the circumference (Top, Right, Bottom,
35     ↪ Left)
36     # This method (Circle Arcs) is more robust for meshing than a single full
37     ↪ circle.
38
39     pc = gmsh.model.geo.addPoint(Xc, Yc, 0, h_fine) # Center
40     p_r = gmsh.model.geo.addPoint(Xc + R, Yc, 0, h_fine)
41     p_t = gmsh.model.geo.addPoint(Xc, Yc + R, 0, h_fine)
42     p_l = gmsh.model.geo.addPoint(Xc - R, Yc, 0, h_fine)
43     p_b = gmsh.model.geo.addPoint(Xc, Yc - R, 0, h_fine)
44
45     # Create 4 arcs to form the circle
46     c1 = gmsh.model.geo.addCircleArc(p_r, pc, p_t)
47     c2 = gmsh.model.geo.addCircleArc(p_t, pc, p_l)
48     c3 = gmsh.model.geo.addCircleArc(p_l, pc, p_b)
49     c4 = gmsh.model.geo.addCircleArc(p_b, pc, p_r)
50
51     # Inner Loop
52     cl_hole = gmsh.model.geo.addCurveLoop([c1, c2, c3, c4])
53
54     # --- 3. Create Surface with Hole ---
55     # The first loop is the boundary, subsequent loops are holes
56     s = gmsh.model.geo.addPlaneSurface([cl_rect, cl_hole])
57
58     # --- 4. Physical Groups (Boundary Conditions) ---
59     # Domain
60     gmsh.model.addPhysicalGroup(2, [s], 1, "Plate")
61
62     # Boundaries
63     gmsh.model.addPhysicalGroup(1, [l4], 2, "FixedLeft") # Left Edge
64     gmsh.model.addPhysicalGroup(1, [l2], 3, "LoadRight") # Right Edge
65     gmsh.model.addPhysicalGroup(1, [c1, c2, c3, c4], 4, "HoleEdge") # Hole boundary
66
67     # Synchronize and Mesh

```

```

66     gmsh.model.geo.synchronize()
67     gmsh.model.mesh.generate(2)
68
69     # Write mesh to file
70     gmsh.write("PlateHole_Q4.msh")
71
72     # Launch GUI to see the result
73     gmsh.fltk.run()
74     gmsh.finalize()

```

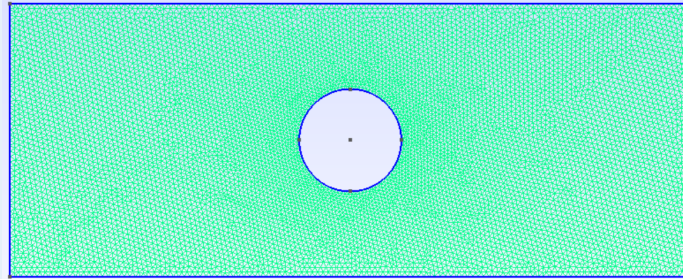


FIGURE 5. Finite element meshing using Julia.

4(a) Steps for finite element meshing using Abaqus.

1) Create a 2D deformable part

- Start Abaqus/CAE → go to the Part module.
- Create a new part:
 - Name: PlateHole_Q4.
 - Modeling space: 2D planar.
 - Type: Deformable.
 - Base feature: Shell (plane stress) or Planar.
 - Approximate size: slightly larger than 2000 mm.

2) Sketch the plate geometry

- In the sketcher, draw a rectangle representing the plate:
 - One corner at (0,0).
 - Opposite corner at (1000,400) mm (length × height).
- In the same sketch, create a circle with:
 - Center at (500,200) mm and Radius = 75 mm.
- Finish the sketch to form the final 2D plate-with-hole geometry.

3) Define material and section (for completeness)

- Go to the Property module.
- Create a material:
 - Name: Steel.
 - Elastic properties: Young's modulus $E=210$ GPa Poisson's ratio $\nu =0.3$.
- Create a solid/plane stress section: Thickness = 2 mm.

- Assign this section to the entire plate part.
- 4) Create assembly
 - Move to the Assembly module.
 - Create a dependent instance of the part (Create Instance).
 - 5) Choose element type
 - Go to the Mesh module.
 - Assign element type:
 - Element family: plane stress (e.g., CPS4/CPS4R).
 - Suitable for 2D stress analysis of thin plate.
 - 6) Seed the edges (global and local refinement)
 - Apply a global seed size (coarse mesh) for the plate
 - Apply a smaller seed size on the hole boundary
 - 7) Generate the mesh
 - With seeding done, Abaqus will generate the 2D finite element mesh for the plate with a hole

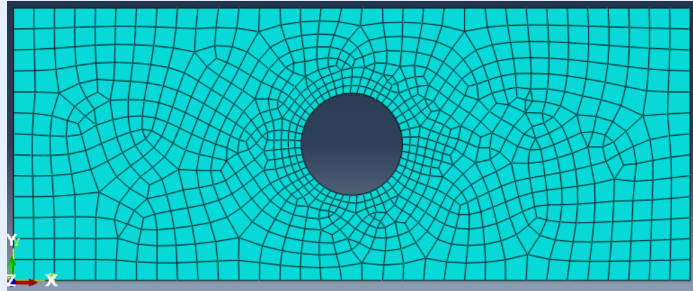


FIGURE 6. Finite element meshing using Abaqus

Solution: 4 (b) Stress distribution.

```

1  using Gridap
2  using GridapGmsh
3
4  # 1. Read the Mesh
5  model = GmshDiscreteModel("PlateHole_Q4.msh")
6
7  # 2. Material Parameters (Steel)
8  const E = 210000.0 # MPa
9  const ν = 0.3      # Poisson's ratio
10 const thickness = 2.0 # mm
11
12 # Lamé parameters for Plane Stress
13 const E = 210000.0 # N/mm^2
14 const ν = 0.3      # Poissons Ratio
15
16 const μ = E/(2*(1+ν))

```

```

17     const λ = (E * ν) / ((1 + ν) * (1 - 2 * ν))
18     const λ_ps = (2 * λ * μ) / (λ + 2 * μ)           # Plane stress correction
19
20     g = VectorValue(100.0, 0.0)                       # Force
21
22     σ(ε) = λ_ps*tr(ε)*one(ε) + 2*μ*ε
23
24     # 3. Define FE Spaces
25     order = 1
26
27     reffe = ReferenceFE(lagrangian,VectorValue{2,Float64},order)
28     V0 = TestFESpace(model,reffe;
29         conformity=:H1,
30         dirichlet_tags=["FixedLeft"],
31         dirichlet_masks=[(true,true)])
32
33     g1 = VectorValue(0.0,0.0)
34     U = TrialFESpace(V0,[g1])
35
36     # 4. Numerical Integration
37     degree = 2*order
38     Ω = Triangulation(model)
39     dΩ = Measure(Ω,degree)
40     Γ_load = BoundaryTriangulation(model, tags = "LoadRight")
41     dΓ_N = Measure(Γ_load,degree)
42
43     # 5. Define Weak Form
44
45     # Internal Work (Stiffness)
46     a(u,v) = ∫( ε(v) ⊙ (σ∘ε(u)) ) * dΩ
47
48     # External Work (Load)
49     l(v) = ∫(v·g) * dΓ_N
50
51     # 6. Solve
52     op = AffineFEOperator(a,l,U,V0)
53     uh = solve(op)
54
55     # 7. Post-Process
56     writevtk(Ω,"results_Q4",
57         cellfields=[
58             "Displacement"=>uh,
59             "Strain"=>ε(uh),

```

```

60      "Stress"=> $\sigma \circ \varepsilon(\text{uh})]$ 
61    )
62

```

4(c) Comparison between the results obtained from Gridap in Julia and Abaqus.

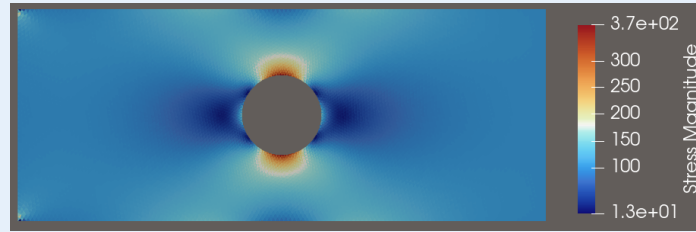


FIGURE 7. Stress distribution using Julia.

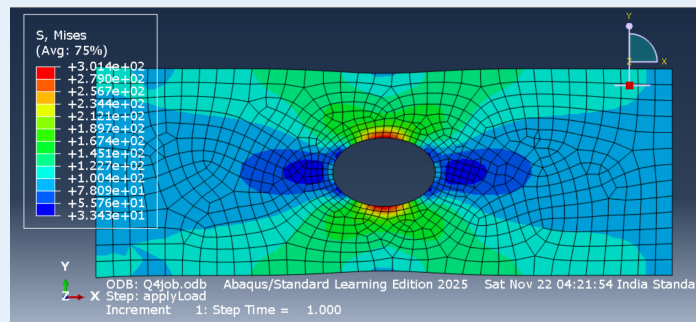


FIGURE 8. Stress distribution using Abaqus.

Question 5. Find out the stress distribution of the cantilever beam of 1000 mm in length and has a 250 mm $\times$ 200 mm rectangular cross-section under the point load of 1000 N at the free end as shown in Figure 2. The section is made up of a concrete having a value of 25000 N/mm² for Young's modulus and a value of 0.2 for Poisson's ratio in the respective fields.

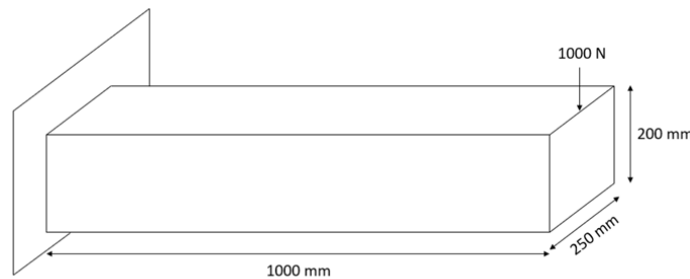


FIGURE 9. Caption

- Provide the steps for finite element meshing both in Julia and Abaqus.
- Mention the governing PDEs that are required to solve for the stress distribution.
- Provide a comparison between the results obtained from Gridap in Julia and Abaqus.

Solution: 5 (a) Steps for finite element meshing using Julia.

```

1      using Gmsh: gmsh
2
3      # Dimensions
4      L = 1000.0    # x-direction (length)
5      W = 250.0     # y-direction (width)
6      H = 200.0     # z-direction (height)
7      h = 10.0      # mesh size
8
9      gmsh.initialize()
10     gmsh.model.add("Cantilever_Q5")
11
12     # --- Create the 8 corner points of the box ---
13     p1 = gmsh.model.geo.addPoint(0, 0, 0, h)
14     p2 = gmsh.model.geo.addPoint(L, 0, 0, h)
15     p3 = gmsh.model.geo.addPoint(L, W, 0, h)
16     p4 = gmsh.model.geo.addPoint(0, W, 0, h)
17
18     p5 = gmsh.model.geo.addPoint(0, 0, H, h)
19     p6 = gmsh.model.geo.addPoint(L, 0, H, h)
20     p7 = gmsh.model.geo.addPoint(L, W, H, h)
21     p8 = gmsh.model.geo.addPoint(0, W, H, h)
22
23     # --- Create lines for bottom face ---
24     l1 = gmsh.model.geo.addLine(p1, p2)
25     l2 = gmsh.model.geo.addLine(p2, p3)
26     l3 = gmsh.model.geo.addLine(p3, p4)
27     l4 = gmsh.model.geo.addLine(p4, p1)
28
29     # --- Create lines for top face ---
30     l5 = gmsh.model.geo.addLine(p5, p6)
31     l6 = gmsh.model.geo.addLine(p6, p7)
32     l7 = gmsh.model.geo.addLine(p7, p8)
33     l8 = gmsh.model.geo.addLine(p8, p5)
34
35     # --- Side edges ---
36     l9  = gmsh.model.geo.addLine(p1, p5)
37     l10 = gmsh.model.geo.addLine(p2, p6)
38     l11 = gmsh.model.geo.addLine(p3, p7)
39     l12 = gmsh.model.geo.addLine(p4, p8)
40
41     # ---- Create surfaces (6 faces of the box) ----
42

```

```

43     # Bottom rectangle
44     cl1 = gmsh.model.geo.addCurveLoop([l1, l2, l3, l4])
45     s1 = gmsh.model.geo.addPlaneSurface([cl1])
46
47     # Top rectangle
48     cl2 = gmsh.model.geo.addCurveLoop([l5, l6, l7, l8])
49     s2 = gmsh.model.geo.addPlaneSurface([cl2])
50
51     # Side faces
52     cl3 = gmsh.model.geo.addCurveLoop([l1, l10, -l5, -l9])
53     s3 = gmsh.model.geo.addPlaneSurface([cl3])
54
55     cl4 = gmsh.model.geo.addCurveLoop([l2, l11, -l6, -l10])
56     s4 = gmsh.model.geo.addPlaneSurface([cl4])
57
58     cl5 = gmsh.model.geo.addCurveLoop([l3, l12, -l7, -l11])
59     s5 = gmsh.model.geo.addPlaneSurface([cl5])
60
61     cl6 = gmsh.model.geo.addCurveLoop([l4, l9, -l8, -l12])
62     s6 = gmsh.model.geo.addPlaneSurface([cl6])
63
64     # ---- Create 3D volume ----
65     sl = gmsh.model.geo.addSurfaceLoop([s1, s2, s3, s4, s5, s6])
66     vol = gmsh.model.geo.addVolume([sl])
67
68     gmsh.model.geo.synchronize()
69
70     # ---- Physical groups ----
71
72     # 3D domain
73     gmsh.model.addPhysicalGroup(3, [vol], 10, "Domain")
74
75     # Fixed End (x = 0 → face s6)
76     gmsh.model.addPhysicalGroup(2, [s6], 2)
77     gmsh.model.setPhysicalName(2, 2, "FixedEnd")
78
79     # Load End (x = L → face s4)
80     gmsh.model.addPhysicalGroup(2, [s4], 3)
81     gmsh.model.setPhysicalName(2, 3, "LoadFace")
82
83     # ---- Mesh ----
84     gmsh.model.mesh.generate(3)
85

```

```

86      # Export mesh
87      gmsh.write("Cantilever_Q5.msh")
88
89      # View mesh
90      gmsh.fltk.run()
91      gmsh.finalize()

```

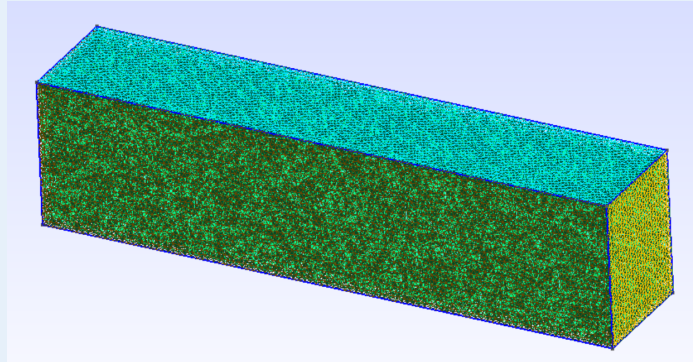


FIGURE 10. Finite element meshing using Julia.

5(a) Steps for finite element meshing using Abaqus.

- 1) Create a 3D deformable solid part
 - Start Abaqus/CAE → go to the Part module.
 - Create a new part:
 - Name: Cantilever_Q5.
 - Modeling space: 3D.
 - Type: Deformable.
 - Base feature: Solid, Extrusion (or Solid).
 - Approximate size: slightly larger than 2000 mm.
- 2) Sketch the cross-section and define the beam length
 - In the sketcher, draw a rectangle representing the cross-section:
 - Width = 250 mm, Height = 200 mm.
 - Finish the sketch.
 - When prompted for extrusion: Extrusion length = 1000 mm (this is the beam length).
- 3) Define material and solid section
 - Go to the Property module.
 - Create a material:
 - Name: Concrete.
 - Elastic properties: Young's modulus $E=25000$ N/mm², Poisson's ratio $\nu=0.2$.
 - Create a solid section (e.g. "BeamSection") and assign the material Concrete.
 - Assign this section to the entire beam part.
- 4) Create assembly
 - Move to the Assembly module.
 - Create a dependent instance of the part (Create Instance).

5) Choose element type and seed the part.

- Go to the Mesh module.
- Assign element type:
 - Choose 3D stress elements, e.g. C3D8 or C3D8R (8-node linear brick, reduced integration).
 - Confirm and apply to the solid region.
 - Choose a suitable global seed size along the beam length and across the cross-section.

6) Generate the mesh

- With seeding done, Abaqus generates the 3D finite element mesh for the cantilever beam.

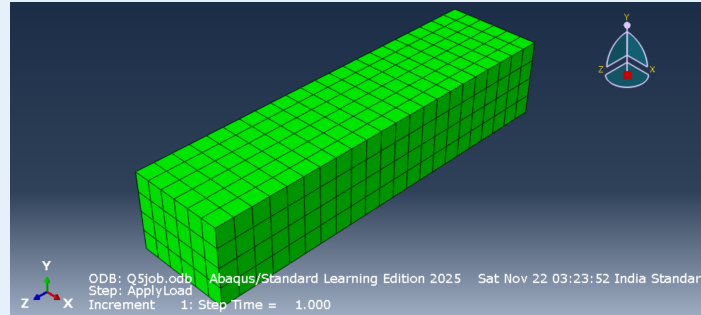


FIGURE 11. Finite element meshing using Abaqus.

Solution: 5 (b) Stress distribution.

```

1  using Gridap
2  using GridapGmsh
3  # 1. Read the Mesh
4  model = GmshDiscreteModel("Cantilever_Q5.msh")
5
6  # 2. Material Parameters (Steel)
7  const E = 25000.0 # MPa
8  const ν = 0.2      # Poisson's ratio
9
10 # Lamé parameters for Plane Stress
11
12 g = VectorValue(0.0,0.0,-1000.0/(250*200)) ## force
13
14 const λ = (E*ν)/((1+ν)*(1-2*ν))
15 const μ = E/(2*(1+ν))
16 σ(ε) = λ*tr(ε)*one(ε) + 2*μ*ε
17
18 # 3. Define FE Spaces
19 order = 1
20
21 reffe = ReferenceFE(lagrangian,VectorValue{3,Float64},order)
22 V0 = TestFESpace(model,reffe;

```

```

23         conformity=:H1,
24         dirichlet_tags=["FixedEnd"],
25         dirichlet_masks=[(true,true,true)])
26
27     g1 = VectorValue(0.0,0.0,0.0)
28     U = TrialFESpace(V0,[g1])
29
30     # 4. Numerical Integration
31     degree = 2*order
32     Ω = Triangulation(model)
33     dΩ = Measure(Ω,degree)
34     Γ_load = BoundaryTriangulation(model, tags = "LoadFace")
35     dΓ_N = Measure(Γ_load,degree)
36
37     # Weak form
38
39     # Internal work (Stiffness component)
40     a(u,v) = ∫( ε(v) ⋅ (σ∘ε(u)) ) * dΩ
41
42     # External Work
43     l(v) = ∫(v⋅g) * dΓ_N
44
45     # Solve
46     op = AffineFEOperator(a,l,U,V0)
47     uh = solve(op)
48
49     # 7. Post-Process
50     writevtk(Ω,"results_Q5",
51         cellfields=[
52             "Displacement"=>uh,
53             "Strain"=>ε(uh),
54             "Stress"=>σ∘ε(uh)]
55     )
56

```

5(c) Comparison between the results obtained from Gridap in Julia and Abaqus.

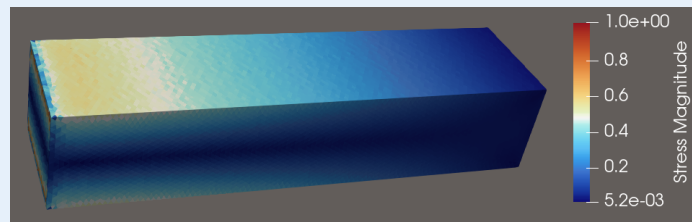


FIGURE 12. Stress distribution using julia

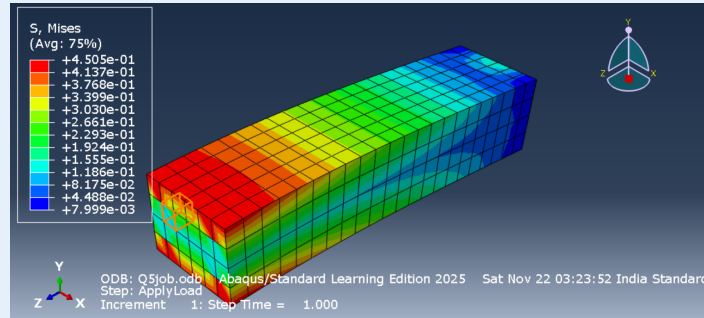


FIGURE 13. Stress distribution using Abaqus

Question 6. Find out the stress distribution of the cantilever beam of 1000 mm in length and has a 250 mm $\times$ 200 mm rectangular cross-section under the uniform distributed load of 1000 N/mm² applied on the top surface as shown in Figure 3. The section is made up of a concrete having a value of 25000 N/mm² for Young's modulus and a value of 0.2 for Poisson's ratio in the respective fields.

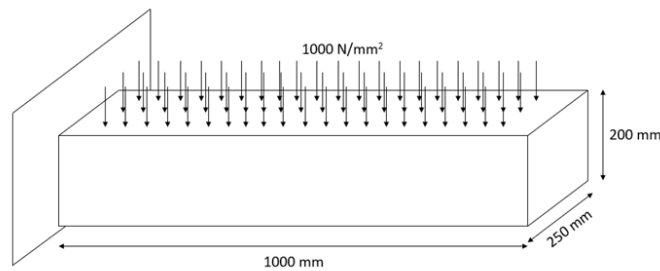


FIGURE 14. Caption

- Provide the steps for finite element meshing both in Julia and Abaqus.
- Mention the governing PDEs that are required to solve for the stress distribution.
- Provide a comparison between the results obtained from Gridap in Julia and Abaqus.

Solution: 6 (a) Steps for finite element meshing using Julia.

```

1      using Gmsh: gmsh
2
3      # Dimensions
4      L = 1000.0 # x-direction (length)
5      W = 250.0  # y-direction (width)
6      H = 200.0  # z-direction (height)
7      h = 10.0   # mesh size
8
9      gmsh.initialize()
10     gmsh.model.add("Cantilever_Q6")
11
12     # --- Create the 8 corner points of the box ---
13     p1 = gmsh.model.geo.addPoint(0, 0, 0, h)
14     p2 = gmsh.model.geo.addPoint(L, 0, 0, h)

```

```

15     p3 = gmsh.model.geo.addPoint(L, W, 0, h)
16     p4 = gmsh.model.geo.addPoint(0, W, 0, h)
17
18     p5 = gmsh.model.geo.addPoint(0, 0, H, h)
19     p6 = gmsh.model.geo.addPoint(L, 0, H, h)
20     p7 = gmsh.model.geo.addPoint(L, W, H, h)
21     p8 = gmsh.model.geo.addPoint(0, W, H, h)
22
23     # --- Create lines for bottom face ---
24     l1 = gmsh.model.geo.addLine(p1, p2)
25     l2 = gmsh.model.geo.addLine(p2, p3)
26     l3 = gmsh.model.geo.addLine(p3, p4)
27     l4 = gmsh.model.geo.addLine(p4, p1)
28
29     # --- Create lines for top face ---
30     l5 = gmsh.model.geo.addLine(p5, p6)
31     l6 = gmsh.model.geo.addLine(p6, p7)
32     l7 = gmsh.model.geo.addLine(p7, p8)
33     l8 = gmsh.model.geo.addLine(p8, p5)
34
35     # --- Side edges ---
36     l9 = gmsh.model.geo.addLine(p1, p5)
37     l10 = gmsh.model.geo.addLine(p2, p6)
38     l11 = gmsh.model.geo.addLine(p3, p7)
39     l12 = gmsh.model.geo.addLine(p4, p8)
40
41     # ---- Create surfaces (6 faces of the box) ----
42
43     # Bottom rectangle
44     cl1 = gmsh.model.geo.addCurveLoop([l1, l2, l3, l4])
45     s1 = gmsh.model.geo.addPlaneSurface([cl1])
46
47     # Top rectangle
48     cl2 = gmsh.model.geo.addCurveLoop([l5, l6, l7, l8])
49     s2 = gmsh.model.geo.addPlaneSurface([cl2])
50
51     # Side faces
52     cl3 = gmsh.model.geo.addCurveLoop([l1, l10, -l5, -l9])
53     s3 = gmsh.model.geo.addPlaneSurface([cl3])
54
55     cl4 = gmsh.model.geo.addCurveLoop([l2, l11, -l6, -l10])
56     s4 = gmsh.model.geo.addPlaneSurface([cl4])
57

```

```

58     c15 = gmsh.model.geo.addCurveLoop([13, 112, -17, -111])
59     s5  = gmsh.model.geo.addPlaneSurface([c15])
60
61     c16 = gmsh.model.geo.addCurveLoop([14, 19, -18, -112])
62     s6  = gmsh.model.geo.addPlaneSurface([c16])
63
64     # ---- Create 3D volume ----
65     s1 = gmsh.model.geo.addSurfaceLoop([s1, s2, s3, s4, s5, s6])
66     vol = gmsh.model.geo.addVolume([s1])
67
68     gmsh.model.geo.synchronize()
69
70     # ---- Physical groups ----
71
72     # 3D domain
73     gmsh.model.addPhysicalGroup(3, [vol], 10, "Domain")
74
75     # Fixed End (x = 0 → face s6)
76     gmsh.model.addPhysicalGroup(2, [s6], 2)
77     gmsh.model.setPhysicalName(2, 2, "FixedEnd")
78
79     # Load End (x = L → face s5)
80     gmsh.model.addPhysicalGroup(2, [s5], 3)
81     gmsh.model.setPhysicalName(2, 3, "LoadFace")
82
83     # ---- Mesh ----
84     gmsh.model.mesh.generate(3)
85
86     # Export mesh
87     gmsh.write("Cantilever_Q6.msh")
88
89     # View mesh
90     gmsh.fltk.run()
91     gmsh.finalize()
92

```

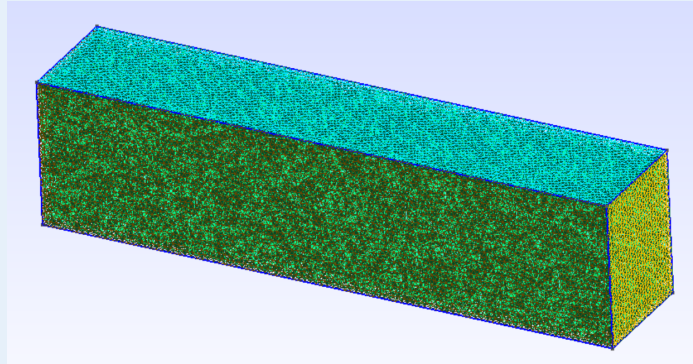


FIGURE 15. Finite element meshing using Julia.

6(a) Steps for finite element meshing using Abaqus.

- Same step as follow in 5.

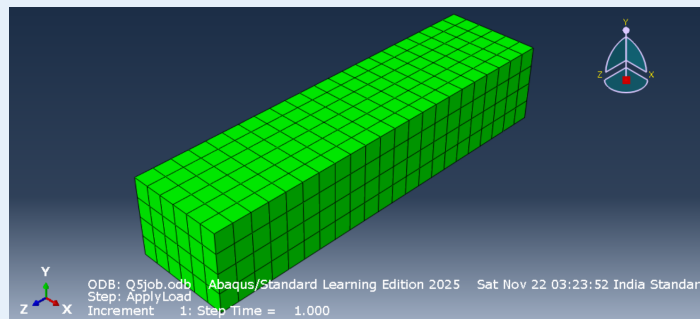


FIGURE 16. Finite element meshing using Abaqus.

Solution: 6 (b) Stress distribution.

```

1      using Gridap
2      using GridapGmsh
3
4      # 1. Read the Mesh
5      model = GmshDiscreteModel("Cantilever_Q6.msh")
6
7      # 2. Material Parameters (Steel)
8      const E = 25000.0 # MPa
9      const ν = 0.2      # Poisson's ratio
10
11     # Lamé parameters for Plane Stress
12
13     g = VectorValue(0.0,0.0,-1000.0)    ## force
14
15     const λ = (E*ν)/((1+ν)*(1-2*ν))
16     const μ = E/(2*(1+ν))
17     σ(ε) = λ*tr(ε)*one(ε) + 2*μ*ε
18

```

```

19      # 3. Define FE Spaces
20      order = 1
21
22      reffe = ReferenceFE(lagrangian,VectorValue{3,Float64},order)
23      V0 = TestFESpace(model,reffe;
24          conformity=:H1,
25          dirichlet_tags=["FixedEnd"],
26          dirichlet_masks=[(true,true,true)])
27
28      g1 = VectorValue(0.0,0.0,0.0)
29      U = TrialFESpace(V0,[g1])
30
31      # 4. Numerical Integration
32      degree = 2*order
33      Ω = Triangulation(model)
34      dΩ = Measure(Ω,degree)
35      Γ_load = BoundaryTriangulation(model, tags = "LoadFace")
36      dΓ_N = Measure(Γ_load,degree)
37
38      # Weak form
39
40      # Internal work (Stiffness component)
41      a(u,v) = ∫( ε(v) ⊙ (σ∘ε(u)) ) * dΩ
42
43      # External Work
44      l(v) = ∫(v·g) * dΓ_N
45
46      # Solve
47      op = AffineFEOperator(a,l,U,V0)
48      uh = solve(op)
49
50      # 7. Post-Process
51      writevtk(Ω,"results_Q6",
52          cellfields=[
53              "Displacement"=>uh,
54              "Strain"=>ε(uh),
55              "Stress"=>σ∘ε(uh)]
56      )

```

6(c) Comparison between the results obtained from Gridap in Julia and Abaqus.

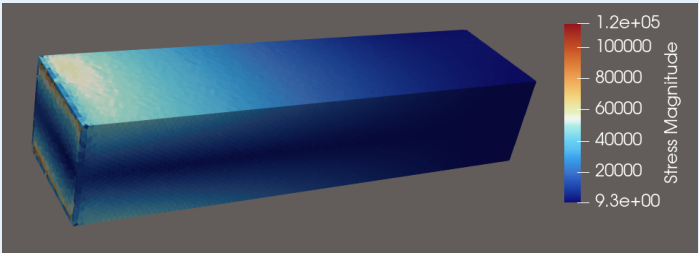


FIGURE 17. Stress distribution using julia

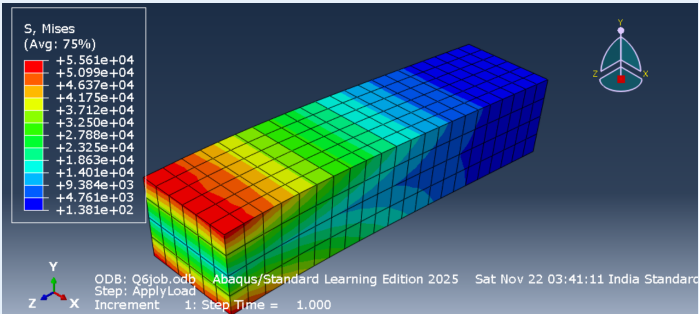


FIGURE 18. Stress distribution using Abaqus

— — — The End — — —